Problem Set 7

Sreshtha Chatterjee 724

2026-03-20

Problem Set 7: Shrinkage Methods

We would be working with the Hitters dataset available in the ISLR library. We have to predict the Salary based on all available variables. First, we make sure that we delete all rows with one or more missing values.

1. We split the data set into a training set and a test set randomly, keeping 100 observations in the test set and the remaining in the training set.

```
set.seed(123)
rm(list=ls())
library(glmnet)

## Warning: package 'glmnet' was built under R version 4.5.3

library(ISLR)

## Warning: package 'ISLR' was built under R version 4.5.2

Hitters = na.omit(Hitters)

test_index = sample(1:nrow(Hitters), 100)

test_data = Hitters[test_index, ]
train_data = Hitters[-test_index, ]
```

2. We train the model using least squares and report the train MSE and test MSE.

```
lm_model = lm(Salary ~ ., data = train_data)

y_hat = predict(lm_model, newdata = train_data)
y_test = predict(lm_model, newdata = test_data)

train_mse = mean((train_data$Salary-y_hat)^2)
train_mse

## [1] 105334.2

test_mse = mean((test_data$Salary-y_test)^2)
test_mse

## [1] 87803.91
```

3.We fit ridge regression models on the training set, with grid values of λ. We report the following:

(i) the dimension of the coefficient matrix so obtained.

(ii) The l_2 norm of the regression coefficients corresponding to the 50th and 60th choice of λ. We comment on the output.

(iii) Test MSE corresponding to these two choices of λ.

```
lambda_grid = 10^seq(5,-2, length = 100)
x_train = model.matrix(Salary~.,train_data)[,-1]
x_test = model.matrix(Salary~.,test_data)[,-1]
head(x_train)
```

```
##                   AtBat Hits HmRun Runs RBI Walks Years CAtBat CHits CHmRun
## -Alan Ashby         315   81     7   24  38    39    14   3449   835     69
## -Alvin Davis        479  130    18   66  72    76     3   1624   457     63
## -Andre Dawson       496  141    20   65  78    37    11   5628  1575    225
## -Alfredo Griffin    594  169     4   74  51    35    11   4408  1133     19
## -Andres Thomas      323   81     6   26  32     8     2    341    86      6
## -Andre Thornton     401   92    17   49  66    65    13   5206  1332    253
##                   CRuns CRBI CWalks LeagueN DivisionW PutOuts Assists
Errors
## -Alan Ashby         321  414    375       1         1     632      43
10
## -Alvin Davis        224  266    263       0         1     880      82
14
## -Andre Dawson       828  838    354       1         0     200      11
3
## -Alfredo Griffin    501  336    194       0         1     282     421
25
## -Andres Thomas       32   34      8       1         1     143     290
19
## -Andre Thornton     784  890    866       0         0       0       0
0
##                   NewLeagueN
## -Alan Ashby                1
## -Alvin Davis               0
## -Andre Dawson              1
## -Alfredo Griffin           0
## -Andres Thomas             1
## -Andre Thornton            0
```

```
head(x_test)
```

```
##                 AtBat Hits HmRun Runs RBI Walks Years CAtBat CHits CHmRun
CRuns
## -Marvell Wynne   288   76     7   34  37    15     4   1644   408     16
198
## -Bill Almon      196   43     7   29  27    30    13   3231   825     36
376
## -Rick Cerone     216   56     4   22  18    15    12   2796   665     43
```

```
266
## -Jack Howell     151   41     4   26  21    19     2    288    68      9
45
## -Spike Owen      528  122     1   67  45    51     4   1716   403     12
211
## -Tom Paciorek    213   61     4   17  22     3    17   4061  1145     83
488
##                CRBI CWalks LeagueN DivisionW PutOuts Assists Errors
NewLeagueN
## -Marvell Wynne  120    113       1         1     203       3      3
1
## -Bill Almon     290    238       1         0      80      45      8
1
## -Rick Cerone    304    198       0         0     391      44      4
0
## -Jack Howell     39     35       0         1      28      56      2
0
## -Spike Owen     146    155       0         1     209     372     17
0
## -Tom Paciorek   491    244       0         1     178      45      4
0
```

```
ridge_model=glmnet(x_train, train_data$Salary, alpha = 0,lambda= lambda_grid)
dim(coef(ridge_model))
```

```
## [1]  20 100
```

(i) Dimension of Coefficient Matrix

```
dim(coef(ridge_model))
```

```
## [1]  20 100
```

(ii) Coefficients for 50th and 60th value of λ respectively

```
cat("50th value\n\n")
```

```
## 50th value
```

```
coef_50 = coef(ridge_model)[-1, 50]
coef_50
```

```
##         AtBat          Hits         HmRun          Runs           RBI
##   -0.56861594    1.57781724    1.45165595    0.26148608    2.54343128
##         Walks         Years        CAtBat         CHits        CHmRun
##    2.88812942   -4.15498762    0.01212119    0.12212743    0.02430066
##         CRuns          CRBI        CWalks       LeagueN     DivisionW
##    0.22110562    0.24061114   -0.27212940   54.66429035 -145.12121490
##       PutOuts       Assists        Errors    NewLeagueN
##    0.35444338    0.26953832   -5.05469505  -14.05949208
```

```
cat("\n\n60th value\n\n")
```

```
##
##
## 60th value
```

```r
coef_60 = coef(ridge_model)[-1, 60]
coef_60
```

```
##         AtBat          Hits         HmRun          Runs           RBI
## -1.408336e+00  3.329840e+00  1.626042e+00 -1.917524e-01  3.107071e+00
##         Walks         Years        CAtBat         CHits        CHmRun
##  4.844059e+00 -7.920993e+00 -5.315477e-03  1.602586e-01 -3.013027e-01
##         CRuns          CRBI        CWalks       LeagueN     DivisionW
##  4.667053e-01  3.938615e-01 -6.029094e-01  5.686263e+01 -1.526506e+02
##       PutOuts       Assists        Errors    NewLeagueN
##  3.918256e-01  3.813593e-01 -5.711395e+00 -2.698770e+01
```

(iii). L2 Norm of Coefficients

```r
l2_norm = function(beta){
  sqrt(sum(beta^2))
}

# l2 norm of 50th lambda
l2_50 = l2_norm(coef_50)
l2_50
```

```
## [1] 155.9135
```

```r
# l2 norm of 60th lambda
l2_60 = l2_norm(coef_60)
l2_60
```

```
## [1] 165.5575
```

(iv). Test MSE for 50th and 60th λ

```r
# Predictions
ridge_pred_50 = predict(ridge_model, s = lambda_grid[50], newx = x_test)
ridge_pred_60 = predict(ridge_model, s = lambda_grid[60], newx = x_test)

# Test MSE for 50th lambda
Test_mse_50 = mean((test_data$Salary - ridge_pred_50)^2)
Test_mse_50
```

```
## [1] 89317.28
```

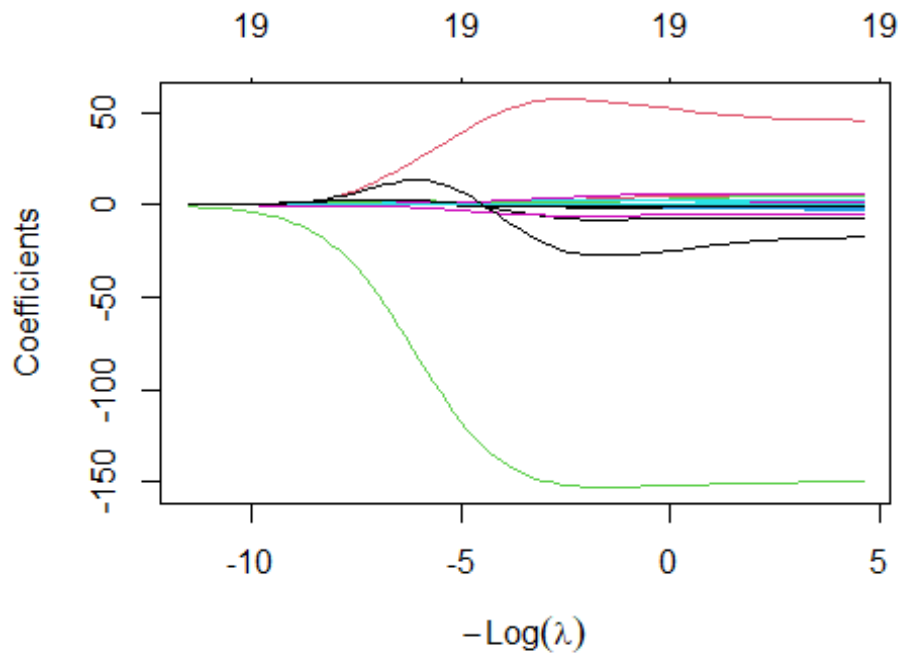
```r
# Test MSE for the 60th lambda
Test_mse_60 = mean((test_data$Salary - ridge_pred_60)^2)
Test_mse_60
```

```
## [1] 87580.18
```

We observe that the test MSE is dropping as λ increases.

4. Make a line plot (for all the predictors) of the standardized coefficients against the grid value of λ chosen by you. Interpret the graph.

```
plot(ridge_model)
```



Interpretation: Here we obtain the values of $-log(\lambda)$ on X-axis. Each of the line plots denote the respective coefficients with respect to $-log(\lambda)$.

- We know that the increasing value of λ the value of $-log(\lambda)$ decreases.
- Hence we can say for the increasing value of λ, the coefficients are shrinking.

5. How would the graph in (4) change if instead of λ you plot on the x axis

the ratio of l2 norm of the ridge regression coefficient to the least square regression coefficient

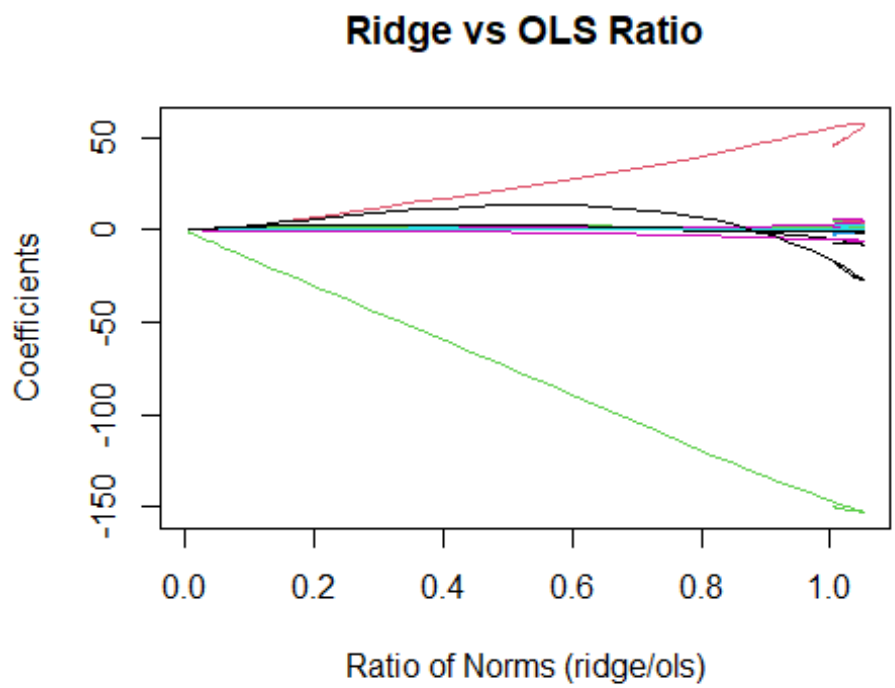
```
beta_ols = coef(lm_model)[-1]

ridge_coefs = as.matrix(coef(ridge_model))[-1, ]

ridge_norms = apply(ridge_coefs, 2, l2_norm)
ols_norm    = l2_norm(beta_ols)

ratio =  ridge_norms / ols_norm
```

```r
matplot(ratio, t(ridge_coefs), type = "l", lty = 1,
        xlab = "Ratio of Norms (ridge/ols)", ylab = "Coefficients",
        main = "Ridge vs OLS Ratio")
```



6. Run the Ridge regression on training data by choosing λ by cross-validation. Find the test MSE for the optimal choice of λ obtained using cross validation.

```r
set.seed(123)
cv_ridge = cv.glmnet(x_train, train_data$Salary, alpha = 0)
cv_ridge

##
## Call:  cv.glmnet(x = x_train, y = train_data$Salary, alpha = 0)
##
## Measure: Mean-Squared Error
##
##     Lambda Index Measure    SE Nonzero
## min    404    69  141709 32690      19
## 1se   7937    37  173528 39657      19

lambda_ridge = cv_ridge$lambda.min
cat("\n\nOptimum Lambda for Ridge Model: ",lambda_ridge,"\n\n")

##
##
## Optimum Lambda for Ridge Model:  404.3208
```

```r
ridge_cv_pred = predict(cv_ridge,"lambda.min", newx = x_test)
ridge_coef = coef(cv_ridge,"lambda.min")

mse_ridge_cv = mean((test_data$Salary - ridge_cv_pred)^2)
non_zero = sum(ifelse(ridge_coef[-1] != 0,1,0))

list(Ridge_Test_MSE = mse_ridge_cv, Nonzero_Coefficients = non_zero)
```

```
## $Ridge_Test_MSE
## [1] 93690.09
##
## $Nonzero_Coefficients
## [1] 19
```

7. Fit a LASSO model on the training set, with λ chosen by cross-validation.Report the test MSE, along with the number of non-zero coefficient estimates.

```r
set.seed(123)
cv_lasso = cv.glmnet(x_train, train_data$Salary, alpha = 1)
cv_lasso
```

```
##
## Call:  cv.glmnet(x = x_train, y = train_data$Salary, alpha = 1)
##
## Measure: Mean-Squared Error
##
##     Lambda Index Measure    SE Nonzero
## min  16.71    29  150268 32062       8
## 1se 141.96     6  179980 41276       2
```

```r
lambda_lasso = cv_lasso$lambda.min
cat("\n\nOptimum Lambda for Lasso Model: ",lambda_lasso,"\n\n")
```

```
##
##
## Optimum Lambda for Lasso Model:  16.70659
```

```r
lasso_cv_pred = predict(cv_lasso,"lambda.min", newx = x_test)
lasso_coef = coef(cv_lasso,"lambda.min")

mse_lasso_cv = mean((test_data$Salary - lasso_cv_pred)^2)
non_zero = sum(ifelse(lasso_coef[-1] != 0,1,0))

list(Lasso_Test_MSE = mse_lasso_cv, Nonzero_Coefficients = non_zero)
```

```
## $Lasso_Test_MSE
## [1] 93914.89
##
## $Nonzero_Coefficients
## [1] 8
```

8. Fit an Elastic Net model on the training set, with λ1 and λ2 chosen by cross-validation. Report the test MSE. Also report the predictors with non-zero coefficients.

```
set.seed(123)
alpha = seq(0, 1, by = 0.1)

cv_errors = c()
models = vector("list")

for(i in 1:length(alpha))
  {
  models[[i]] = cv.glmnet(x_train, train_data$Salary, alpha = alpha[i])
  cv_errors[i] = min(models[[i]]$cvm)
  }

best_id = which.min(cv_errors)
best_alpha = alpha[best_id]
best_model = models[[best_id]]

lambda_elastic = best_model$lambda.min
cat("\n\nOptimum Lambda for Elastic Net Model: ",lambda_elastic,"\n\n")

##
##
## Optimum Lambda for Elastic Net Model:  95.60123

pred_elastic = predict(best_model,"lambda.min", newx = x_test)
mse_elastic  = mean((test_data$Salary - pred_elastic)^2)

coef_elastic = coef(best_model,"lambda.min")
nz_vars = rownames(coef_elastic)[coef_elastic[,1] != 0][-1]

list(Elastic_Test_MSE = mse_elastic, Non_Zero_Predictors = nz_vars)

## $Elastic_Test_MSE
## [1] 91957.09
##
## $Non_Zero_Predictors
##  [1] "Hits"      "HmRun"     "RBI"       "Walks"     "CAtBat"    "CHits"
##  [7] "CHmRun"    "CRuns"     "CRBI"      "LeagueN"   "DivisionW" "PutOuts"
## [13] "Assists"   "Errors"
```

9. Compare the test MSE 's of least squares regression, Ridge regression, LASSO and ELASTIC NET and comment on how accurately we can predict the Salary of a baseball player.

Test MSE values:

- Least Squares: 87803.91
- Ridge: 93690.09
- LASSO: 93914.89
- Elastic Net: 91957.09

We observe that: - The test MSE is least for the OLS method. - Out of the regularized methods, Elastic Net performs the best. - LASSO and Elastic Net generate simpler models by reducing the number of predictor, which makes the models more interpretable. - Overall for pure accuracy, OLS should be prefered while regularization can used for dimensionality reduction and superior interpretability.